Project Progress Report

Overview

This game project is going to be an interactive mystery game, so in the core features we need interactive elements, we are going to need dialog features, and a system to handle evidence and such. For this part we have some basic menus, some rough interaction scripts, mouse visual scripts, the beginnings of a complex dialog system, and some other fun stuff.

---

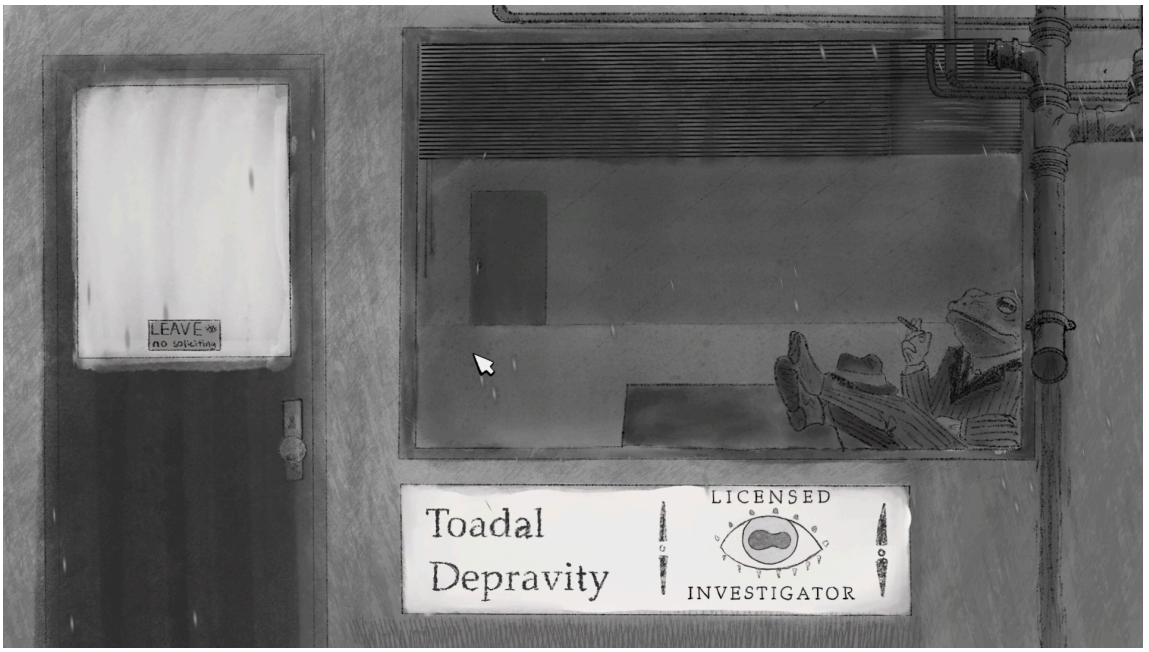
1. Implemented Features

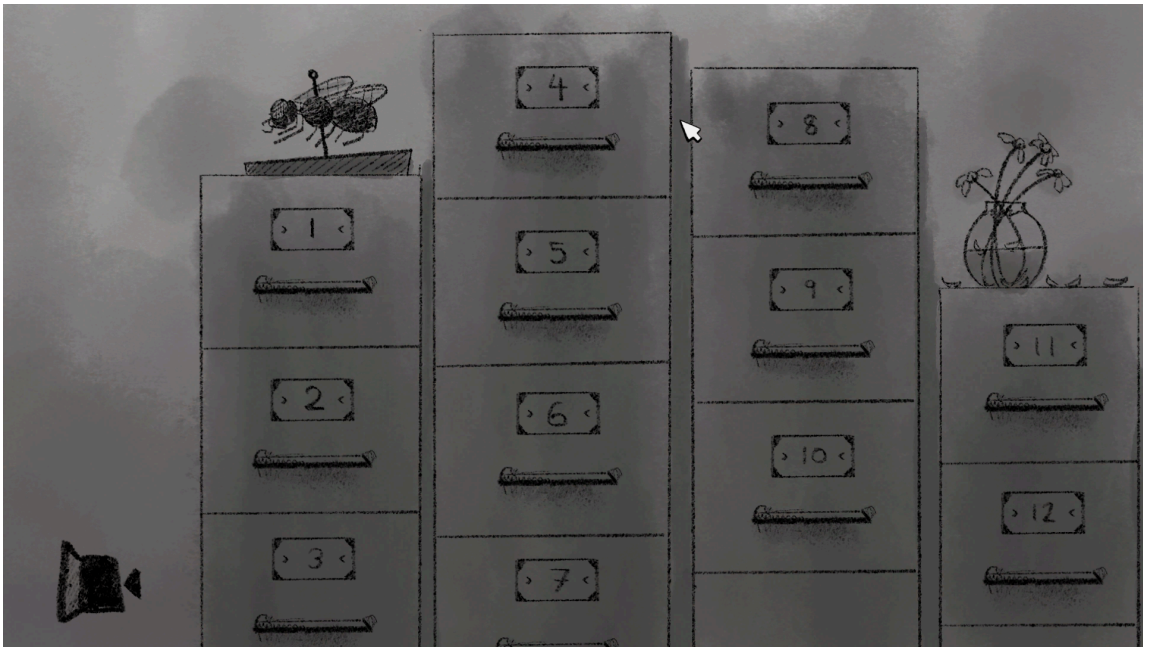
1.1 Core Navigation & Menus

I first established the foundational scenes and interactive UI elements:

- **Main Menu:** The entry point.
- **Office Menu:** The hub where players can review their cases and navigate ongoing investigations.
- **Case Selection:** A dedicated screen allowing players to access available mysteries.

Showcase: *(Screenshots from recent dev builds)*





Many of the interactive visual elements in these scenes rely on specialized MonoBehaviours, such as `HoverGrayscaleSprite`. This shader effect gives buttons and sprites a visual response when the player inspects them with the mouse.

```
[DisallowMultipleComponent]
[RequireComponent(typeof(SpriteRenderer))]
[RequireComponent(typeof(Collider2D))]
public sealed class HoverGrayscaleSprite : MonoBehaviour
{
    [Header("Hover Animation")]
```

```
    private const float transitionSeconds = 0.12f;
    private const float hoverScaleMultiplier = 1.06f;

    [Header("Grayscale")]
    [SerializeField]
    private Shader grayscaleShader;

    private const float grayscaleWhenNotHovered = 1f;
    private const float grayscaleWhenHovered = 0f;

    private void Awake()
    {
        spriteRenderer = GetComponent<SpriteRenderer>();
        col2D = GetComponent<Collider2D>();
        baseScale = transform.localScale;

        // Ensure custom shader is attached for grayscale effect
        if (grayscaleShader == null)
            grayscaleShader = Shader.Find("Toads/GrayscaleSprite");

        runtimeMaterial = new Material(grayscaleShader);
        spriteRenderer.material = runtimeMaterial;

        currentGrayscale = grayscaleWhenNotHovered;
        ApplyGrayscale(currentGrayscale);
    }

    private void Update()
    {
        UpdateHoverState();

        float smoothTime = Mathf.Max(0.0001f, transitionSeconds);

        // Smoothly enlarge and colorize the sprite
        transform.localScale = Vector3.SmoothDamp(transform.localScale, targetScale, ref
scaleVelocity, smoothTime);
        currentGrayscale = Mathf.SmoothDamp(currentGrayscale, targetGrayscale, ref
grayscaleVelocity, smoothTime);
        ApplyGrayscale(currentGrayscale);
    }

    private void SetHovered(bool hovered)
    {
        targetScale = hovered ? baseScale * hoverScaleMultiplier : baseScale;
        targetGrayscale = hovered ? grayscaleWhenHovered : grayscaleWhenNotHovered;
    }
}
```

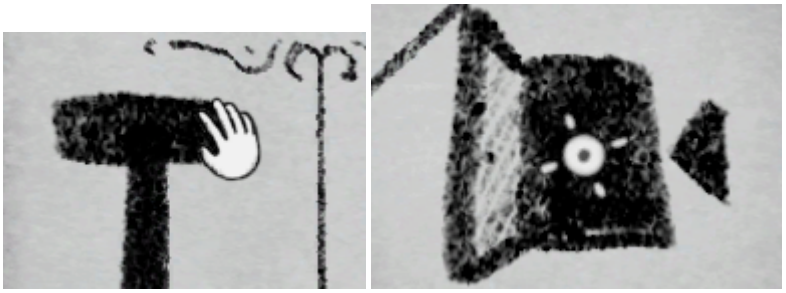
The (`SceneTransitionManager.cs`) moves players across these different hub menus using a generated fuzzy black fade-out screen.

```
[DisallowMultipleComponent]
public class SceneTransitionManager : MonoBehaviour
{
    public static SceneTransitionManager Instance { get; private set; }

    [SerializeField]
    private float transitionDuration = 3.5f;

    [SerializeField]
    private Texture2D fuzzyTexture;

    private void Awake()
    {
        // Singleton setup with persistence across scenes
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }

        Instance = this;
        DontDestroyOnLoad(gameObject);

        SetupTransitionUI();
    }

    // Generates a fuzzy/noise overlay screen and hooks it to a CanvasGroup
    private void SetupTransitionUI() { ... }
}
```

1.2 Interactions & Minigames

The core of the mechanics relies heavily on interaction:

- **Mouse Grab & Target Systems:** Allowing players to inspect and interact.
- **Interactive Mini-games:** Specifically, the early iteration of the Roulette minigame.



Showcase:

1.3 Dialogue & State Management (The `ConditionHandler`)

To manage complex dialogue branching across investigations, I developed a system for tracking game states natively in Unity.

How it works: The `ConditionHandler` stores and tracks bool states (conditions) to dictate which dialogue branches to display.

```
/// <summary>
/// Manages setting, checking, and resetting dialog conditions using PlayerPrefs.
/// </summary>
public static class ConditionHandler
{
    private const string Prefix = "DLG_COND_";
    private const string KeysTracker = "DLG_COND_KEYS";

    public static void SetCondition(string conditionName, bool value = true)
    {
        if (string.IsNullOrEmpty(conditionName)) return;

        PlayerPrefs.SetInt(Prefix + conditionName, value ? 1 : 0);
```

```
        TrackKey(conditionName);
        PlayerPrefs.Save();
    }

    public static bool HasCondition(string conditionName)
    {
        if (string.IsNullOrEmpty(conditionName)) return true; // No requirement
        return PlayerPrefs.GetInt(Prefix + conditionName, 0) == 1;
    }
}
```

How it fails: It might not be super scalable and it will get very hard to work with complex dialog. It also is not optimal with reading and writing.

1.4 Dialog Flow and JSON Parsing

Beyond simply tracking conditions, the dialog system itself is a Node-based system that creates branching conversations. It has the `BaseDialogController` and the `DialogParser`.

How it works: The system uses JSON files to store dialog branches, options, and what conditions are set for each statement. The `DialogParser` pulls these JSON tree structures (such as `ExampleDialog.json`) and converts them into interconnected `DialogNode` objects at runtime.

```
[System.Serializable]
public class DialogNode
{
    public string speakerName;
    [TextArea(3, 5)]
    public string text;

    // Condition tags
    public string requiresCondition; // Condition that must be true to process
    public string setsCondition;     // Condition set to true when displayed

    public List<DialogOption> options = new List<DialogOption>();
    public DialogNode nextNode;
}
```

The system gets the conditions in real-time when populating choices:

```
if (ConditionHandler.HasCondition(opt.requiresCondition))
{
    validOptions.Add(opt);
}
```

How it fails:

- **JSON Linkages:** If a `targetId` is misspelled in the JSON file, the entire dialogue branch breaks entirely and leads to a dead end without warnings.

---

2. Next Steps

2.1 Upcoming Features

Following our roadmap, the immediate next steps will focus on **Phase 3: Dialogue & Interrogation**:

1. **Case Evidence** A way to collect and display evidence (rather dependant on my ability to create art at a slow rate)
2. **Evidence Presenting Mechanic:** Tying the `ConditionHandler` tightly with the inventory to allow presenting items directly into dialogue branches correctly.
3. **Global GameStateManager Upgrade:** Moving away from static helper classes into an serialized Game State object to manage rules across scenes.

2.2 Making Current Features Failure-Resistant

1. **Refactor `ConditionHandler` for Memory Safety:** Instead of calling `PlayerPrefs` actively, load the state into an active Dictionary at runtime. Only call `Save()` passively (e.g., end of scene, autosaves). This avoids constant disk writes.